



VISUAL PROGRAMMING

JDBC

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

OUTLINES

- **Introduction**
- **JDBC**
- **The Architecture of JDBC**
- **The JDBC Interfaces**
- **Developing JDBC Programs**
- **Processing Statements**
- **The execute, executeQuery, and executeUpdate Methods**
- **The Prepared Statement**
- **TableView**

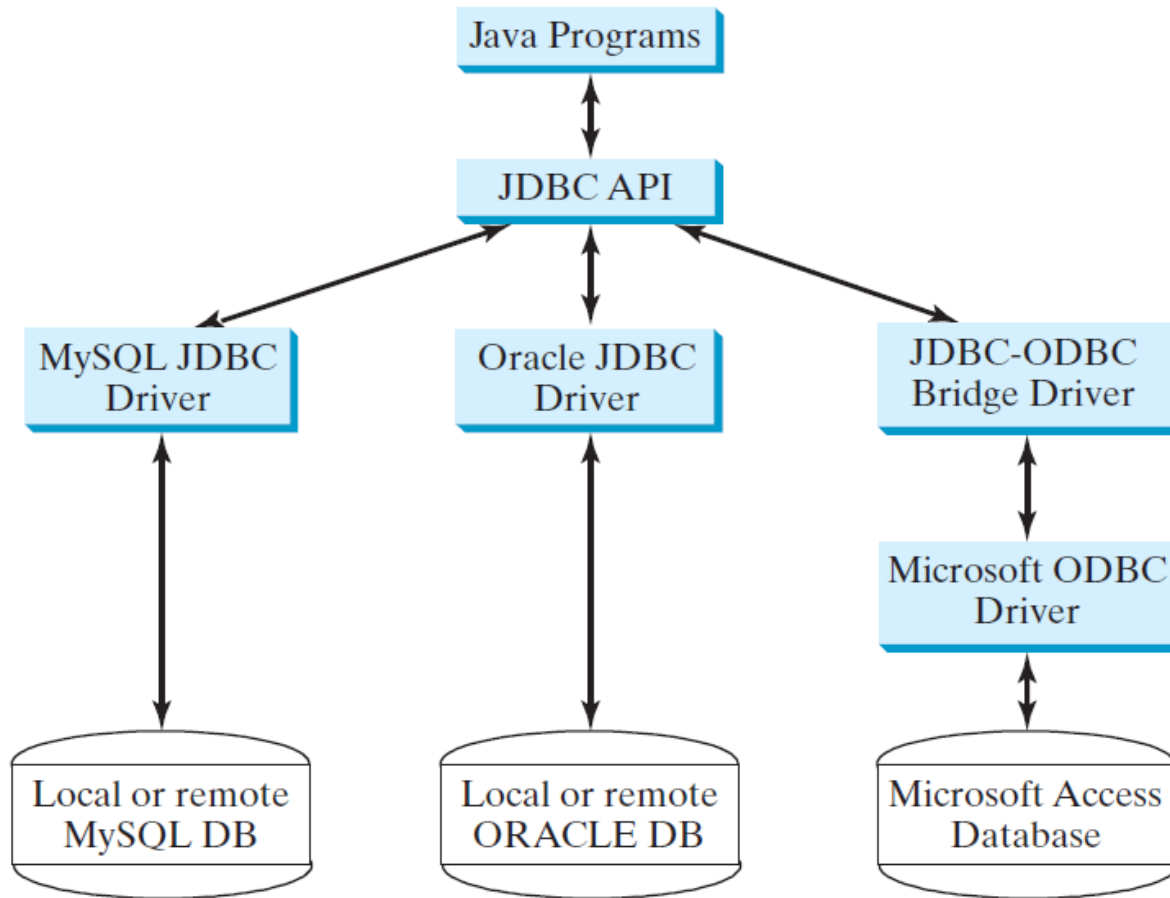
INTRODUCTION

- **JavaFX empowers us to create visually appealing user interfaces, but the heart of many applications lies in data manipulation.**
- **To achieve this, we integrate Java applications with databases.**
- **In this lecture, we'll explore the seamless integration of Java applications with databases, leveraging the power of JDBC.**
- **With JDBC, we connect, interact, and manipulate data effectively.**

JDBC

- The Java Database Connectivity (JDBC) API provides universal data access from the Java programming language. Using the JDBC API, you can access virtually any data source, from relational databases to spreadsheets and flat files.

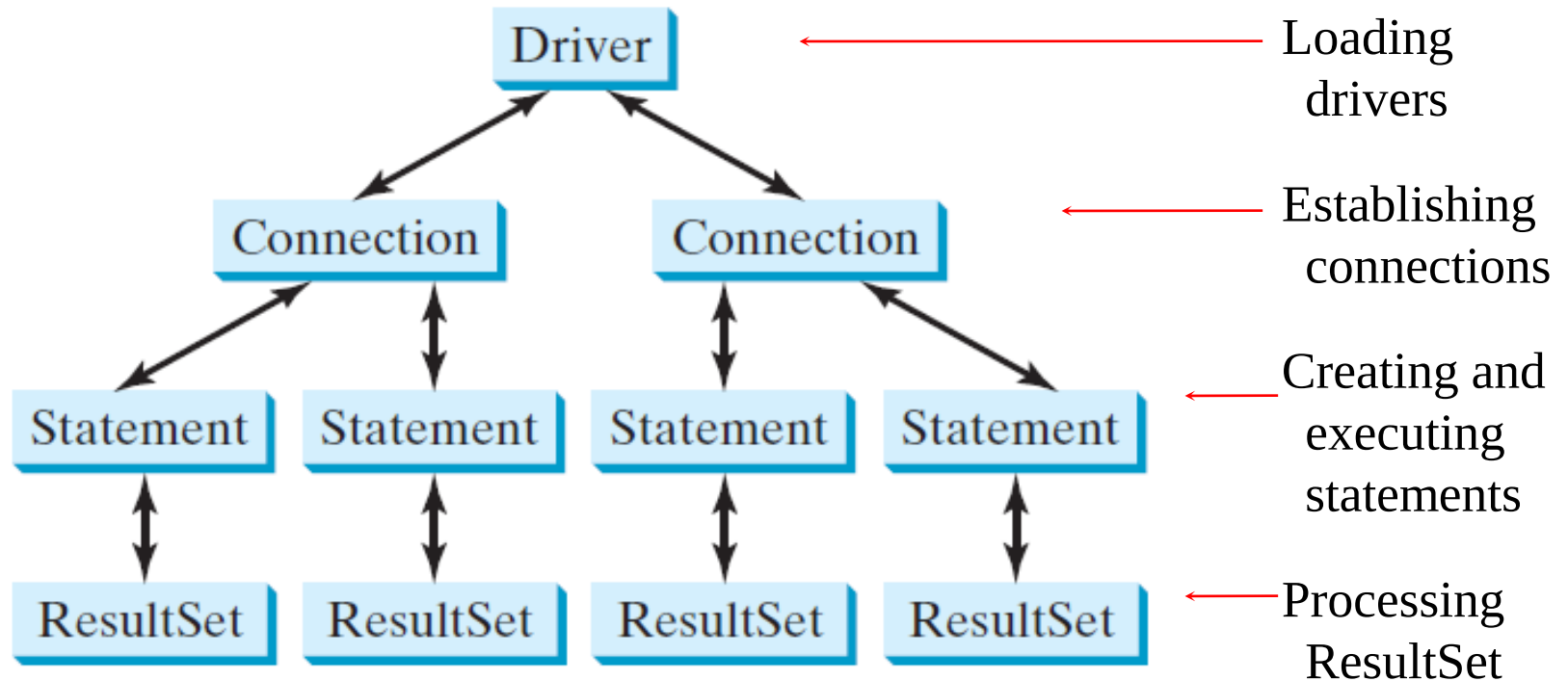
THE ARCHITECTURE OF JDBC



THE ARCHITECTURE OF JDBC

- **JDBC API:** It provides various methods and interfaces for easy communication with the database.
 - **Examples:** Drivers, DriverManager, Statement, Connection, CallableStatement, PreparedStatement, ResultSet, SQL data.
- **DriverManager:** It loads a database-specific driver in an application to establish a connection with a database.
- **JDBC drivers:** To interact with a data source with the help of the JDBC, one needs a JDBC driver which conveniently interacts with the respective data source.

THE JDBC INTERFACES



DEVELOPING JDBC PROGRAMS

Loading drivers

Establishing connections

Creating and executing statements

Processing ResultSet

- The Statement to load a driver:

```
Class.forName("JDBCDriverClass");
```

- A **driver** is a class.
- For example:

Database	Driver Class	Source
Access	sun.jdbc.odbc.JdbcOdbcDriver	Already in JDK
MySQL	com.mysql.jdbc.Driver	Website
Oracle	oracle.jdbc.driver.OracleDriver	Website

DEVELOPING JDBC PROGRAMS

Loading drivers

Establishing connections

Creating and executing statements

Processing ResultSet

- **The statement to establishing connections:**

```
Connection connection = DriverManager.getConnection(databaseURL);
```

- **Example for MySQL:**

```
Connection connection =  
DriverManager.getConnection("jdbc:mysql://localhost/test");
```

Database	URL Pattern
Access	jdbc:odbc:datasource
MySQL	jdbc:mysql://hostname/dbname
Oracle	jdbc:oracle:thin:@hostname:port#:oracleDBSID

DEVELOPING JDBC PROGRAMS

Loading drivers	• Creating statement: Statement statement = connection.createStatement(); • Executing statement (for update, delete, insert): statement.executeUpdate("create table Temp (col1 char(5), col2 char(5))"); • Executing statement (for select): // Select the columns from the Student table ResultSet resultSet = statement.executeQuery("select firstName, mi, lastName from Student where lastName = 'Smith'");
Establishing connections	
Creating and executing statements	
Processing ResultSet	

DEVELOPING JDBC PROGRAMS

Loading drivers	Executing statement (for select): <i>// Select the columns from the Student table</i>
Establishing connections	<code>ResultSet resultSet = stmt.executeQuery("select firstName, mi, lastName from Student where lastName = 'Smith'");</code>
Creating and executing statements	Processing ResultSet (for select): <i>// Iterate through the result and print the student names</i>
Processing ResultSet	<code>while (resultSet.next()) System.out.println(resultSet.getString(1) + " " + resultSet.getString(2) + ". " + resultSet.getString(3));</code>

FULL EXAMPLE

SimpleJdbc

```
import java.sql.*;
public class SimpleJdbc {
    public static void main(String[] args)
        throws SQLException, ClassNotFoundException {
        // Load the JDBC driver
        Class.forName("com.mysql.jdbc.Driver");
        System.out.println("Driver loaded");

        // Establish a connection
        Connection connection = DriverManager.getConnection
            ("jdbc:mysql://localhost/test");
        System.out.println("Database connected");
    }
}
```

FULL EXAMPLE (CONT.)

SimpleJdbc

// Create a statement

```
Statement statement = connection.createStatement();
```

// Execute a statement

```
ResultSet resultSet = statement.executeQuery
```

```
("select firstName, mi, lastName from Student where lastName "  
  + " = 'Smith'");
```

FULL EXAMPLE (CONT.)

SimpleJdbc

```
// Iterate through the result and print the student names
while (resultSet.next())
    System.out.println(resultSet.getString(1) + "\t" +
        resultSet.getString(2) + "\t" + resultSet.getString(3));

// Close the connection
connection.close();
}
}
```

PROCESSING STATEMENTS

- Once a connection to a particular database is established, it can be used to send SQL statements from your program to the database.
- JDBC provides the **Statement**, **PreparedStatement**, and **CallableStatement** interfaces to facilitate sending statements to a database for execution and receiving execution results from the database.

The **execute**, **executeQuery**, and **executeUpdate** Methods

- The methods for executing SQL statements are **execute**, **executeQuery**, and **executeUpdate**.
- Each of which accepts a string containing a SQL statement as an argument.
- This string is passed to the database for execution.

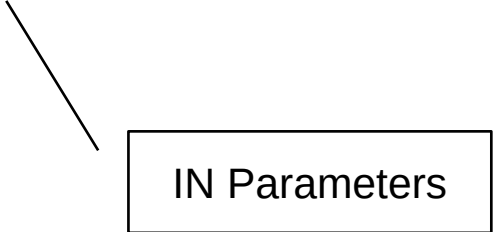
The **execute**, **executeQuery**, and **executeUpdate** Methods

- The **execute** method should be used if the execution produces **multiple** result sets, **multiple** update counts, or a combination of result sets and update counts.
- The **executeQuery** method should be used if the execution produces a **single** result set, such as the SQL select statement.
- The **executeUpdate** method should be used if the statement results in a single update count or no update count, such as a SQL INSERT, DELETE, UPDATE, or DDL statement.

PreparedStatement

- The **PreparedStatement** interface is designed to execute **dynamic** SQL statements and SQL-stored procedures with IN parameters.
- These SQL statements and stored procedures are **precompiled for efficient use** when repeatedly executed.
- **Example:**

```
Statement pstmt = connection.prepareStatement("insert into Student  
(firstName, mi, lastName) + values (?, ?, ?)");  
pstmt.setString(1, "Iyas");  
pstmt.setString(2, "A.");  
pstmt.setString(3, "Khalil");
```

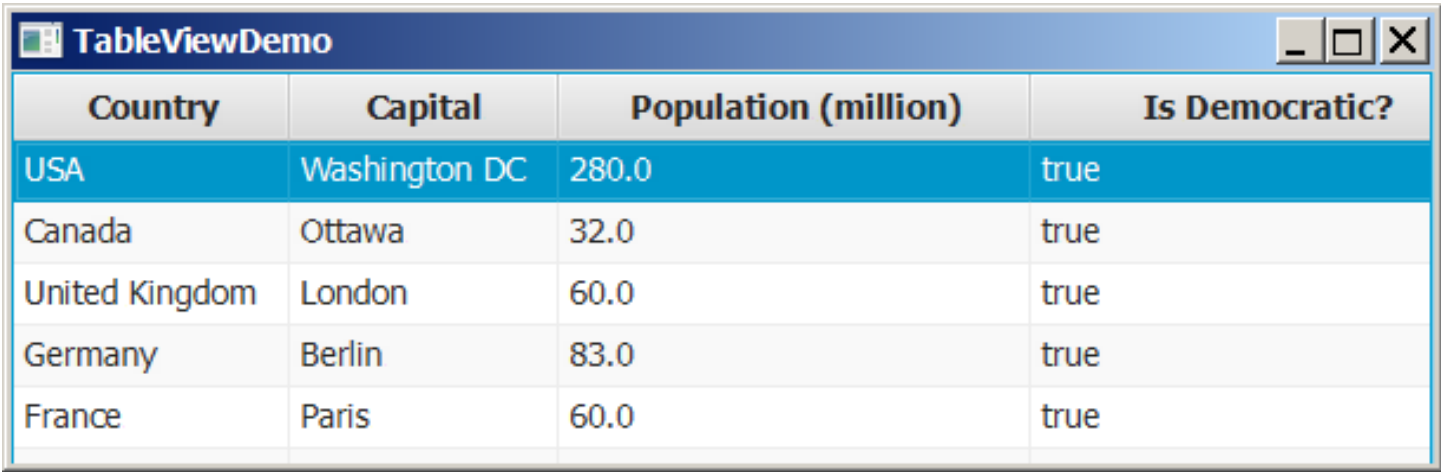


IN Parameters

Table View

TableView

You can display tables using the TableView class.

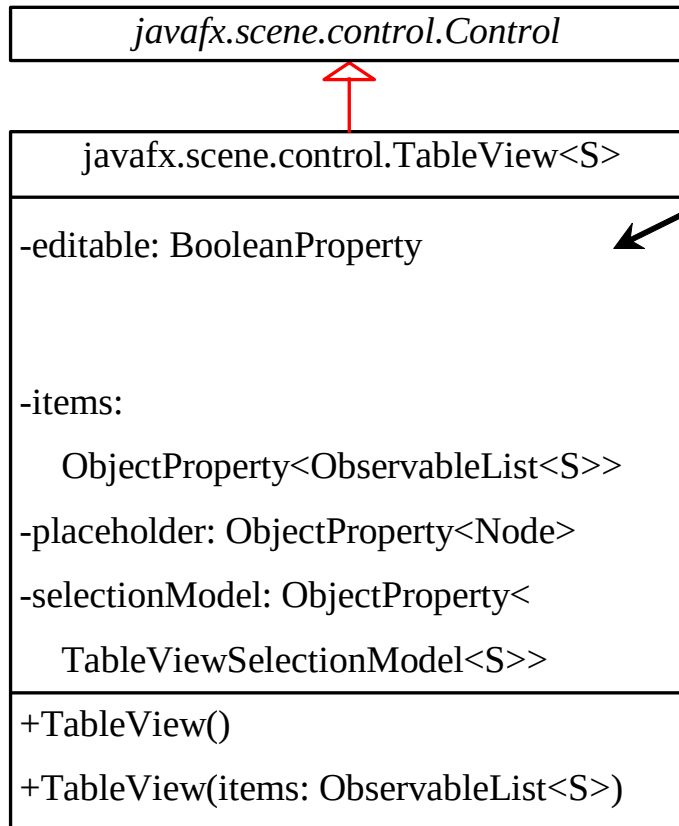


Country	Capital	Population (million)	Is Democratic?
USA	Washington DC	280.0	true
Canada	Ottawa	32.0	true
United Kingdom	London	60.0	true
Germany	Berlin	83.0	true
France	Paris	60.0	true

TableView

- The JavaFX **TableView** class uses a set of related classes to do its job.
- These classes are:
 - TableColumn
 - TableRow
 - TableCell
 - TablePosition
 - TableViewFocusModel
 - TableViewSelectionModel

The TableView Class



The `getter` and `setter` methods for property values and a `getter` for property itself are provided in the class, but omitted in the UML diagram for brevity.

Specifies whether this `TableView` is editable. For a cell to be editable, `TableView`, `TableColumn`, and `TableCell` for the cell should all be true.

The data model for the `TableViee`.

This `Node` is shown when table has no contents.

Specifies single or multiple selections.

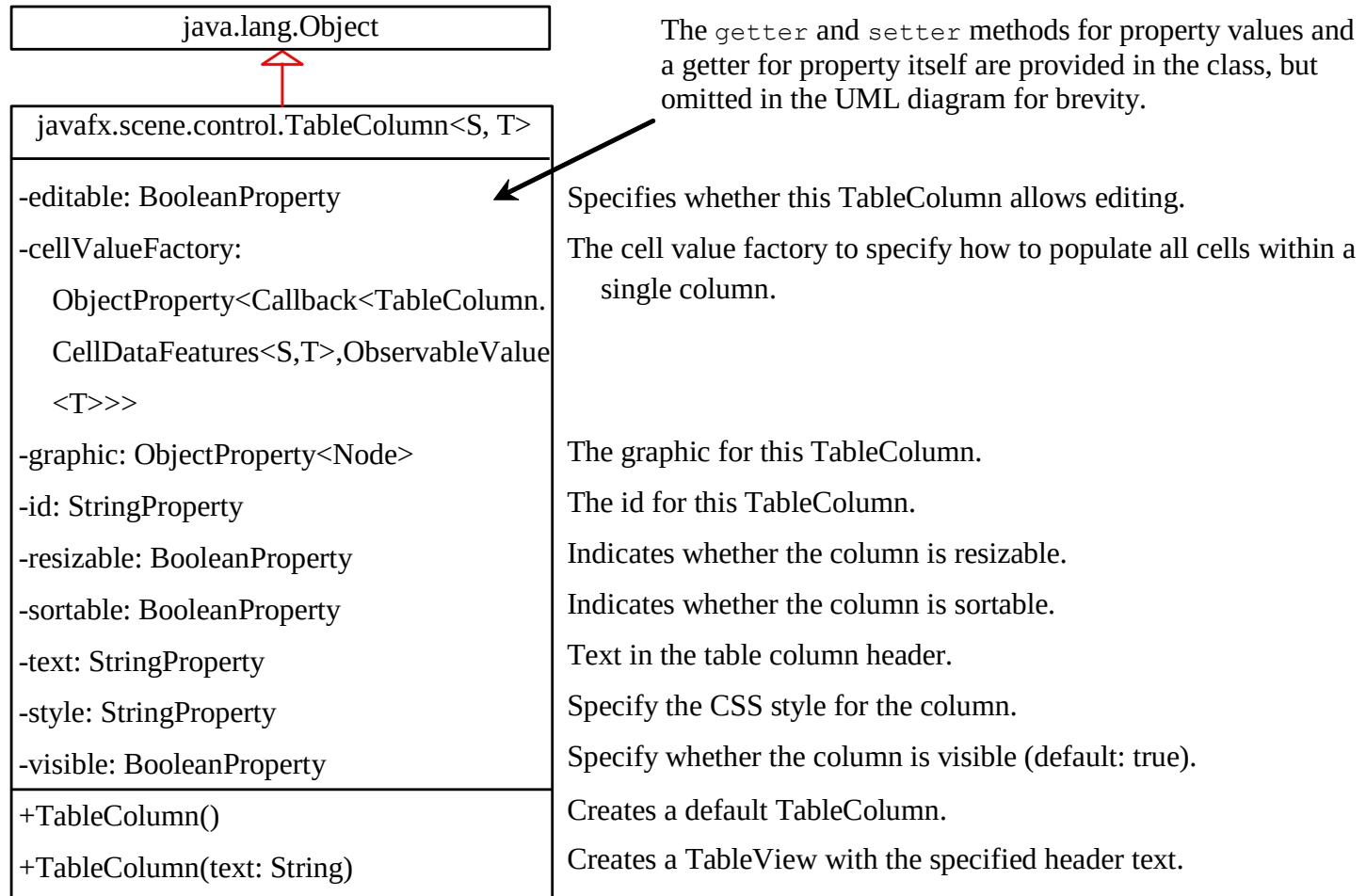
Creates a default `TableView` with no content.

Creates a default `TableView` with the specified content.

TableColumn

- To create a **TableView** you need to add one or more **TableColumn** instances to the **TableView** instance.
- A **TableColumn** represents a vertical column of values.

The TableColumn Class



TableColumn Cell Value Factory

- A **TableColumn** must have a **cell value factory** set on it.
- The cell value factory extracts the value to be displayed in each cell (on each row) in the column.
- You can use **PropertyValueFactory** to extract the values from your data class.
- You can use **MapValueFactory** to extract the values from a **Map**.

TableView Example

Data Class

```
public class Student{  
    private StringProperty name;  
    private IntegerProperty Id;  
    private ObjectProperty<Date> dateOfBirth;  
  
    // constructor  
  
    // getters, setters, and property methods  
}
```

TableView Example

Create TableView

```
// Create TableView
```

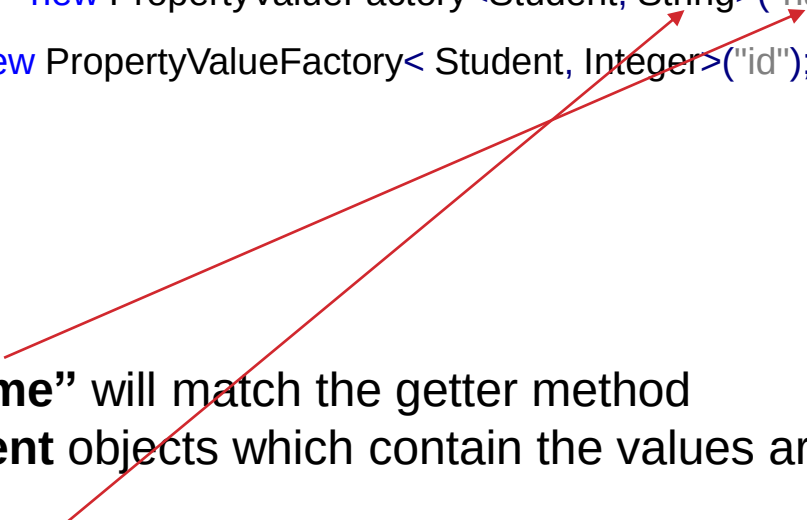
```
TableView<Student> tableView = new TableView<>();
```

TableView Example

Create Cell Factories

// Create Cell Factories

```
PropertyValueFactory nameFactory = new PropertyValueFactory<Student, String>("name");  
PropertyValueFactory idFactory = new PropertyValueFactory< Student, Integer>("id");
```



Note:

- The property name “**name**” will match the getter method **getName()** of the **Student** objects which contain the values are displayed on each row.
- And the property type “**String**” will match the getter method **return type**.

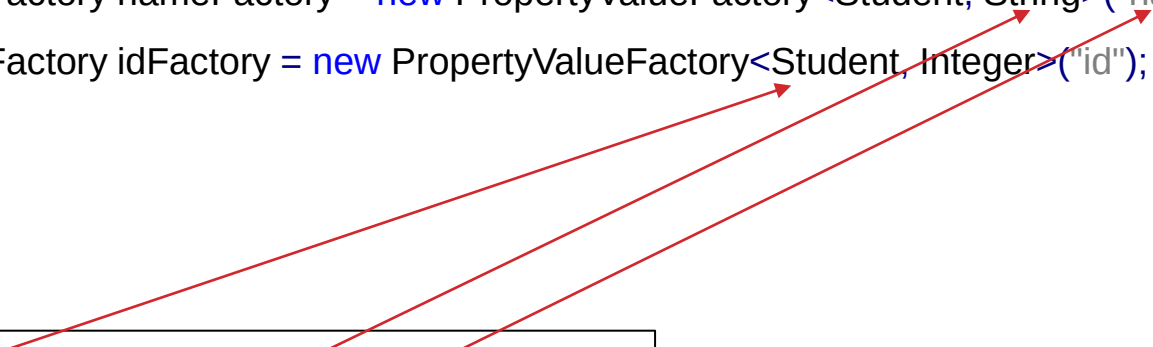
TableView Example

Create Cell Factories

// Create Cell Factories

```
PropertyValueFactory nameFactory = new PropertyValueFactory<Student, String>("name");  
PropertyValueFactory idFactory = new PropertyValueFactory<Student, Integer>("id");
```

```
public class Student{  
    private StringProperty name;  
    ...  
    public String getName(){  
        return name.get();  
    }  
    ...  
}
```



TableView Example

Create Columns

```
// Create TableColumns
```

```
TableColumn nameColumn = new TableColumn("Name");  
    nameColumn.setMinWidth(100);  
    nameColumn.setCellValueFactory(nameFactory);
```

```
TableColumn idColumn = new TableColumn("Id");  
    idColumn.setMinWidth(50);  
    idColumn.setCellValueFactory(idFactory);
```

TableView Example

Final steps

// Add the columns to the TableView

```
tableView.getColumns().addAll(nameColumn, idColumn);
```

// Add data to the Table View

```
tableView.getItems().add(new Student("Ahmad", 125));
```

ANY QUESTIONS?